

Natural Language Processing - 50 Interview Questions with Answers

NLP Fundamentals (Questions 1-10)

Q1. What is Natural Language Processing (NLP)?

Answer: NLP is a branch of AI that enables computers to understand, interpret, and generate human language.

Key areas:

- **Understanding:** Extracting meaning from text
- **Generation:** Creating human-like text
- **Translation:** Converting between languages
- **Analysis:** Sentiment, entities, relationships

Applications:

- Chatbots and virtual assistants
- Machine translation (Google Translate)
- Sentiment analysis
- Text summarization
- Question answering
- Named Entity Recognition

Q2. Explain tokenization and its types.

Answer: Breaking text into smaller units (tokens).

Types:

1. Word Tokenization:

```
"Hello world!" → ["Hello", "world", "!"]
```

2. Sentence Tokenization:

```
"Hi. How are you?" → ["Hi.", "How are you?"]
```

3. Subword Tokenization:

- **BPE** (Byte Pair Encoding): Merge frequent pairs
- **WordPiece:** Used by BERT
- **SentencePiece:** Language-agnostic

```
"unhappiness" → ["un", "happi", "ness"]
```

4. Character Tokenization:

```
"cat" → ["c", "a", "t"]
```

Challenges:

- Contractions: "don't" → "do" + "n't"
- Punctuation handling
- Unknown words
- Multiple languages

Q3. What is stemming vs lemmatization?

Answer:

Stemming: Crude chopping of word endings

```
running → run  
leaves → leav  
better → better
```

Algorithms: Porter, Snowball **Pros:** Fast **Cons:** May not be real words

Lemmatization: Returns dictionary base form (lemma)

```
running → run  
leaves → leaf  
better → good  
am/is/are → be
```

Requires: POS tagging, dictionary **Pros:** Real words, accurate **Cons:** Slower

When to use:

- **Stemming:** Search engines, IR
- **Lemmatization:** Text analysis, ML models

Q4. Explain stop words and their importance.

Answer: Common words with little semantic value.

Examples: a, an, the, is, are, in, on, at, to, from

Removal benefits:

- Reduces dimensionality
- Focuses on meaningful words
- Improves efficiency
- Better search results

When NOT to remove:

- Sentiment analysis ("not good" ≠ "good")
- Question answering
- Machine translation
- Modern transformers (they handle stop words well)

Python example:

```
from nltk.corpus import stopwords
stop_words = set(stopwords.words('english'))
filtered = [w for w in words if w not in stop_words]
```

Q5. What is Part-of-Speech (POS) tagging?

Answer: Assigning grammatical categories to words.

Tags:

- **NN:** Noun (cat, dog)
- **VB:** Verb (run, eat)
- **JJ:** Adjective (happy, blue)
- **RB:** Adverb (quickly, very)
- **DT:** Determiner (the, a)
- **IN:** Preposition (in, on)

Example:

```
"The quick brown fox jumps"
[('The', 'DT'), ('quick', 'JJ'), ('brown', 'JJ'),
 ('fox', 'NN'), ('jumps', 'VBZ')]
```

Applications:

- Lemmatization
- Named Entity Recognition
- Information extraction
- Machine translation

Methods:

- Rule-based
- HMM (Hidden Markov Models)
- CRF (Conditional Random Fields)
- Neural networks (BiLSTM, Transformers)

Q6. What is Named Entity Recognition (NER)?

Answer: Identifying and classifying named entities in text.

Entity types:

- **PERSON:** Barack Obama, John Smith
- **ORGANIZATION:** Google, UN
- **LOCATION:** New York, Paris
- **DATE:** January 1, 2024
- **TIME:** 3:00 PM
- **MONEY:** \$100, €50
- **PERCENTAGE:** 50%

Example:

```
"Apple Inc. was founded by Steve Jobs in California"  
[Apple Inc. - ORG]  
[Steve Jobs - PERSON]  
[California - LOC]
```

Approaches:

- Rule-based (regex, dictionaries)
- Statistical (CRF, HMM)
- Neural (BiLSTM-CRF, BERT)

Libraries: spaCy, Stanford NER, Flair

Applications:

- Information extraction
- Question answering
- Search engines
- Content recommendation

Q7. Explain the bag-of-words (BoW) model.

Answer: Represents text as unordered collection of words, ignoring grammar and word order.

Process:

1. Create vocabulary from corpus
2. Count word occurrences in each document
3. Represent as vector

Example:

```
Doc1: "I love NLP"
Doc2: "I love AI"

Vocabulary: [I, love, NLP, AI]
Doc1: [1, 1, 1, 0]
Doc2: [1, 1, 0, 1]
```

Advantages:

- Simple to implement
- Works well for classification
- Fast

Disadvantages:

- Loses word order
- No semantic meaning
- Sparse vectors
- No context

Improvements:

- TF-IDF weighting
- N-grams
- Word embeddings

Q8. What is TF-IDF?

Answer: Term Frequency-Inverse Document Frequency: weights words by importance.

Formula:

```
TF-IDF(t,d) = TF(t,d) × IDF(t)

TF(t,d) = Count of term t in document d / Total terms in d

IDF(t) = log(Total documents / Documents containing t)
```

Intuition:

- **High TF:** Word appears often in document (important locally)
- **High IDF:** Word is rare across corpus (discriminative)
- **Low IDF:** Common words like "the" (less important)

Example:

```
Doc1: "cat cat dog"
Doc2: "dog mouse"

TF-IDF for "cat" in Doc1:
TF = 2/3 = 0.67
IDF = log(2/1) = 0.30
TF-IDF = 0.67 × 0.30 = 0.20
```

Use cases:

- Document search
- Feature extraction
- Keyword extraction
- Document similarity

Q9. What are word embeddings?

Answer: Dense vector representations of words that capture semantic meaning.

Key properties:

- Words with similar meanings have similar vectors
- Vector arithmetic: king - man + woman $\approx$ queen
- Fixed dimensional (typically 100-300)

Popular methods:**1. Word2Vec:**

- CBOW: Predict word from context
- Skip-gram: Predict context from word

2. GloVe (Global Vectors):

- Matrix factorization on co-occurrence

3. FastText:

- Character n-grams
- Handles unknown words

4. Contextual (modern):

- ELMo: BiLSTM-based
- BERT: Transformer-based
- Context-dependent embeddings

Advantages over BoW:

- Captures semantics
- Dense (not sparse)

- Pre-trained available
- Transfer learning

Q10. Explain sequence-to-sequence models.

Answer: Architecture for mapping input sequences to output sequences.

Architecture:

Encoder → Context Vector → Decoder

Components:

1. Encoder:

- Processes input sequence
- Creates fixed-size context vector
- Usually LSTM/GRU

2. Decoder:

- Generates output sequence
- Conditioned on context vector
- Autoregressive generation

Applications:

- Machine translation
- Text summarization
- Question answering
- Dialogue systems
- Speech recognition

Limitations:

- Fixed-size bottleneck
- Long sequences problematic
- Information loss

Solution: Attention mechanism (addresses bottleneck)

Text Processing (Questions 11-20)

Q11. What is text normalization?

Answer: Converting text to standard/consistent form.

Techniques:

1. Lowercasing:

```
"Hello World" → "hello world"
```

2. Removing punctuation:

```
"Hello, world!" → "Hello world"
```

3. Removing numbers:

```
"Price is $100" → "Price is"
```

4. Expanding contractions:

```
"don't" → "do not"
```

5. Removing URLs/emails:

```
"Visit http://example.com" → "Visit"
```

6. Unicode normalization:

```
"café" (multiple representations) → single form
```

7. Spelling correction:

```
"helo" → "hello"
```

When to apply: Depends on task

- Keep capitalization for NER
- Keep punctuation for sentiment

Q12. Explain n-grams and their applications.

Answer: Contiguous sequences of n items from text.

Types:

1. Unigram (n=1): Individual words

```
"I love NLP" → ["I", "love", "NLP"]
```

2. Bigram (n=2): Pairs

```
"I love NLP" → ["I love", "love NLP"]
```

3. Trigram (n=3): Triples

```
"I love NLP" → ["I love NLP"]
```

Character n-grams:

```
"cat" → ["ca", "at"] (bigrams)
```

Applications:

- Language modeling
- Text generation
- Feature extraction
- Spelling correction
- Text classification

Trade-offs:

- Higher n: More context, sparse
- Lower n: Less context, common

Skip-grams: Non-contiguous n-grams

Q13. What is cosine similarity?

Answer: Measures similarity between two vectors based on angle.

Formula:

$$\begin{aligned}\text{cosine_similarity}(A, B) &= (A \cdot B) / (||A|| \times ||B||) \\ &= \sum(A_i \times B_i) / (\sqrt{\sum(A_i^2)} \times \sqrt{\sum(B_i^2)})\end{aligned}$$

Range: -1 to 1

- 1: Identical direction
- 0: Orthogonal (unrelated)
- -1: Opposite direction

Example:

```
A = [1, 2, 3]
B = [2, 4, 6]
# B is 2x A, so cosine = 1 (same direction)

C = [1, 0, 0]
D = [0, 1, 0]
# Orthogonal, so cosine = 0
```

Use in NLP:

- Document similarity
- Semantic search
- Clustering
- Recommendation

Advantage: Ignores magnitude, focuses on direction

Q14. What is sentiment analysis?

Answer: Determining emotional tone/opinion in text.

Levels:

1. Document-level: Entire document sentiment

```
"This movie was great!" → Positive
```

2. Sentence-level: Per sentence

```
"Food was great but service was poor."
→ Positive, Negative
```

3. Aspect-level: Specific aspects

```
"Great food [Positive], poor service [Negative]"
```

Approaches:

1. Lexicon-based:

- Use sentiment dictionaries (VADER, SentiWordNet)
- Count positive/negative words

2. Machine Learning:

- Train classifier (Naive Bayes, SVM, LR)
- Features: BoW, TF-IDF, n-grams

3. Deep Learning:

- LSTM, CNN, BERT
- Learn representations automatically

Challenges:

- Sarcasm: "Great, another rainy day!"
- Negation: "not good"
- Context: "The movie was sick" (good vs bad)

Q15. Explain text classification pipeline.

Answer:

Steps:

1. Data Collection:

- Gather labeled text data
- Ensure balanced classes

2. Preprocessing:

- Tokenization
- Lowercasing
- Stop word removal
- Stemming/lemmatization

3. Feature Extraction:

- BoW, TF-IDF
- Word embeddings
- Contextual embeddings (BERT)

4. Model Selection:

- Traditional: Naive Bayes, SVM, Logistic Regression
- Deep Learning: CNN, LSTM, Transformers

5. Training:

- Split data (train/val/test)
- Train model
- Hyperparameter tuning

6. Evaluation:

- Accuracy, Precision, Recall, F1
- Confusion matrix
- Cross-validation

7. Deployment:

- API endpoint
- Monitoring
- Retraining

Example tasks:

- Spam detection
- Topic classification
- Sentiment analysis

Q16. What is language modeling?

Answer: Predicting probability of word sequences.

Types:

1. Statistical (N-gram):

$$P(\text{"I love NLP"}) = P(\text{"I"}) \times P(\text{"love"} | \text{"I"}) \times P(\text{"NLP"} | \text{"love"})$$

- Markov assumption
- Limited context

2. Neural (RNN/LSTM):

- Longer context
- Better generalization

3. Transformer-based (GPT, BERT):

- Full sequence context
- State-of-the-art

Evaluation: Perplexity (lower is better)

$$\text{Perplexity} = \exp(-1/N \times \sum \log P(w_i))$$

Applications:

- Text generation
- Speech recognition
- Machine translation
- Autocomplete

Masked Language Modeling (BERT):

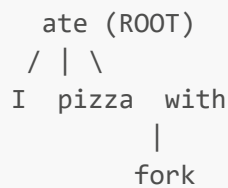
- Predict masked words
- Bidirectional context

Q17. What is dependency parsing?

Answer: Analyzing grammatical structure showing word dependencies.

Example:

"I ate pizza with fork"

**Dependency relations:**

- **nsubj**: Nominal subject (I → ate)
- **dobj**: Direct object (pizza → ate)
- **prep**: Preposition (with → ate)
- **pobj**: Object of preposition (fork → with)

vs Constituency Parsing:

- Dependency: Word-to-word relations
- Constituency: Phrase structure (NP, VP, PP)

Applications:

- Information extraction
- Question answering
- Semantic role labeling

Tools: spaCy, Stanford Parser

Q18. Explain attention mechanism in NLP.

Answer: Allows model to focus on relevant parts of input when generating output.

Problem it solves:

- Seq2seq bottleneck
- Long sequence information loss

How it works:

1. Compute attention scores between decoder state and all encoder states

2. Apply softmax to get attention weights
3. Weighted sum of encoder states = context vector
4. Use context vector for prediction

Formula:

```
score( $h_t, \bar{h}_s$ ) =  $h_t^T W_a \bar{h}_s$   
attention_weights = softmax(scores)  
context =  $\Sigma(\text{attention\_weights} \times \text{encoder\_states})$ 
```

Types:

- **Additive** (Bahdanau)
- **Multiplicative** (Luong)
- **Self-attention** (Transformer)
- **Multi-head attention**

Benefits:

- Better long sequence handling
- Interpretability (visualize attention)
- State-of-the-art performance

Q19. What is coreference resolution?

Answer: Identifying which expressions refer to the same entity.

Example:

```
"John went to the store. He bought milk."  
  
"He" → "John" (coreference)
```

More complex:

```
"Mary told Sarah that she won the award."  
  
"she" → Mary? Sarah? (ambiguous)
```

Types:**1. Pronoun resolution:**

- he, she, it, they → entity

2. Noun phrase coreference:

- "the president" → "Barack Obama"

3. Zero anaphora:

- Implicit reference (common in Japanese)

Approaches:

- Rule-based (syntactic constraints)
- Statistical (mention-pair models)
- Neural (end-to-end with BERT)

Applications:

- Information extraction
- Question answering
- Text summarization
- Machine translation

Tools: Stanford CoreNLP, spaCy, AllenNLP

Q20. What is text summarization?

Answer: Creating shorter version preserving key information.

Types:

1. Extractive:

- Select important sentences
- No new text generation
- Methods: TextRank, graph-based, ML scoring

Example:

```
Original: [S1: intro] [S2: key point] [S3: detail] [S4: conclusion]
Summary: [S2] [S4]
```

2. Abstractive:

- Generate new sentences
- Paraphrase and condense
- Methods: Seq2seq, Transformers (BART, T5, Pegasus)

Example:

```
Original: "The meeting was held on Monday. It lasted 2 hours."
Summary: "Monday's meeting lasted 2 hours."
```

Evaluation:

- **ROUGE**: N-gram overlap with reference
- **BLEU**: Precision-based
- **BERTScore**: Semantic similarity
- Human evaluation

Challenges:

- Factual consistency
- Coverage vs conciseness
- Handling long documents

Advanced NLP (Questions 21-30)

Q21. What is transfer learning in NLP?

Answer: Using pre-trained models on new tasks with limited data.

Process:**1. Pre-training:**

- Train on large corpus
- Learn general language representations
- Unsupervised (language modeling)

2. Fine-tuning:

- Adapt to specific task
- Train on labeled task data
- Few epochs needed

Benefits:

- Less data needed
- Better performance
- Faster training
- Lower compute cost

Pre-trained models:

- **BERT**: Masked language model
- **GPT**: Causal language model
- **T5**: Text-to-text
- **RoBERTa**: Optimized BERT

Fine-tuning strategies:

- Full fine-tuning
- Feature extraction (freeze base)
- Adapter layers

- LoRA (Low-Rank Adaptation)

Example:

```
BERT (110M params, pre-trained on books+wiki)
↓ Fine-tune
Sentiment Classifier (few thousand examples)
```

Q22. Explain BERT and its innovations.

Answer: Bidirectional Encoder Representations from Transformers

Key innovations:**1. Bidirectional context:**

- Sees left and right context
- vs GPT (left-only)

2. Masked Language Modeling (MLM):

- Mask 15% of tokens
- Predict masked tokens

```
"The [MASK] sat on the mat" → predict "cat"
```

3. Next Sentence Prediction (NSP):

- Predict if sentence B follows A
- (Later removed in RoBERTa)

Architecture:

- Transformer encoder layers
- BERT-base: 12 layers, 768 hidden, 12 heads
- BERT-large: 24 layers, 1024 hidden, 16 heads

Input representation:

```
[CLS] + Tokens + [SEP] + Segment + Position embeddings
```

Variants:

- **RoBERTa:** Optimized training
- **ALBERT:** Parameter sharing
- **DistilBERT:** Smaller, faster
- **ELECTRA:** Discriminative pre-training

Usage:

```
from transformers import BertModel
model = BertModel.from_pretrained('bert-base-uncased')
```

Q23. What is the difference between BERT and GPT?

Answer:

Aspect	BERT	GPT
Architecture	Encoder-only	Decoder-only
Attention	Bidirectional	Causal (left-to-right)
Training	Masked LM	Next token prediction
Best for	Understanding tasks	Generation tasks
Input	Can see full sequence	Only previous tokens
Tasks	Classification, NER, QA	Text generation, completion
Context	Bidirectional context	Unidirectional context

BERT pre-training:

```
"The [MASK] is blue" → predict "sky"
Uses left and right context
```

GPT pre-training:

```
"The sky is" → predict "blue"
Uses only left context
```

When to use:

- **BERT:** Sentiment analysis, NER, classification
- **GPT:** Story generation, chat, completion

Both: Can be fine-tuned for various tasks

Q24. What is machine translation and its approaches?

Answer: Automatic translation between languages.

Approaches:**1. Rule-based:**

- Manual linguistic rules
- Dictionaries
- Cons: Doesn't scale, rigid

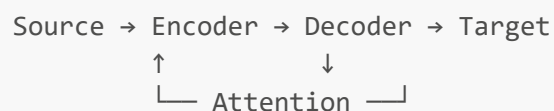
2. Statistical (SMT):

- Learn from parallel corpora
- Phrase-based models
- Language models
- Used until ~2016

3. Neural (NMT):

- **Seq2seq** with attention
- **Transformer** (current SOTA)
- End-to-end learning

Architecture (Transformer):



Challenges:

- Rare words
- Long sentences
- Idioms
- Context
- Low-resource languages

Evaluation:

- **BLEU**: N-gram precision
- **METEOR**: Alignment + synonyms
- Human evaluation

Examples: Google Translate, DeepL, M2M-100

Q25. Explain zero-shot and few-shot learning in NLP.

Answer:

Zero-shot Learning:

- No training examples for task
- Model generalizes from description

Example:

```
Prompt: "Translate English to French: Hello"  
Output: "Bonjour"  
(without any translation training)
```

Few-shot Learning:

- Few examples (1-10) in prompt
- Model learns from examples

Example:

```
Prompt:  
English: Hello → French: Bonjour  
English: Goodbye → French: Au revoir  
English: Thank you → French: ?  
  
Output: Merci
```

In-context Learning:

- Learning from context without parameter updates
- Emergent ability in large models (GPT-3, GPT-4)

Requirements:

- Large pre-trained models
- Good prompting
- Task instruction clarity

Benefits:

- No fine-tuning needed
- Fast deployment
- Works across many tasks

Limitations:

- Less accurate than fine-tuning
- Context window limits
- Expensive inference

Q26. What is question answering (QA)?

Answer: Systems that answer questions based on text.

Types:**1. Extractive QA:**

- Extract answer span from context
- Example: SQuAD dataset

Context: "Paris is the capital of France"
Question: "What is the capital of France?"
Answer: "Paris" (extracted)

2. Abstractive QA:

- Generate answer (not necessarily in context)

Context: "Paris is the capital. It has the Eiffel Tower."
Question: "Describe Paris"
Answer: "Paris is France's capital with the Eiffel Tower"

3. Open-domain QA:

- Answer from large corpus or web
- Requires retrieval + reading

4. Closed-domain QA:

- Specific domain (medical, legal)

Approaches:

1. Retrieval-based:

- Retrieve relevant documents
- Extract answer

2. Knowledge-based:

- Use knowledge graphs

3. Neural:

- BERT for extractive QA
- T5, GPT for abstractive
- RAG (Retrieval Augmented Generation)

Evaluation: Exact Match, F1 score

Q27. Explain dialogue systems and chatbots.

Answer: Systems that converse with users in natural language.

Types:

1. Rule-based:

- Pattern matching
- Decision trees
- Limited, predictable

2. Retrieval-based:

- Select response from corpus
- Match user input to stored responses
- Can't generate new responses

3. Generative:

- Generate responses
- Seq2seq, GPT
- More flexible but can be unpredictable

Components:

1. Natural Language Understanding (NLU):

- Intent detection
- Entity extraction

```
"Book a flight to NYC" → Intent: book_flight, Entity: NYC
```

2. Dialogue Management:

- Track conversation state
- Decide next action

3. Natural Language Generation (NLG):

- Generate response

Frameworks:

- Rasa
- Dialogflow (Google)
- Wit.ai (Meta)
- Amazon Lex

Evaluation:

- Task completion
- User satisfaction
- Response quality

Q28. What is named entity linking (entity disambiguation)?

Answer: Linking entity mentions to knowledge base entries.

Problem:

"I visited Paris yesterday"

Paris → Paris, France?

Paris → Paris, Texas?

Paris → Paris Hilton?

Process:**1. Mention detection:**

- Identify entity mentions (NER)

2. Candidate generation:

- Find possible KB entities

3. Disambiguation:

- Select correct entity using context

Features for disambiguation:

- Context similarity
- Entity popularity
- String matching
- Type constraints

Knowledge bases:

- Wikipedia/DBpedia
- Wikidata
- YAGO
- Freebase

Applications:

- Knowledge graph construction
- Information extraction
- Semantic search
- Question answering

Tools: DBpedia Spotlight, spaCy with EntityLinker

Q29. What is word sense disambiguation (WSD)?

Answer: Determining which sense of word is used in context.

Problem:

```
"I went to the bank"  
bank → river bank? financial bank?  
  
"The bat flew away"  
bat → animal? baseball bat?
```

Approaches:

1. Knowledge-based:

- Use dictionaries (WordNet)
- Overlap between definitions and context

2. Supervised:

- Train classifier on sense-annotated data
- Features: context words, POS, syntax

3. Unsupervised:

- Cluster word contexts
- Each cluster = sense

4. Neural:

- Contextual embeddings (BERT)
- Different contexts → different vectors

WordNet senses:

```
bank.n.01: sloping land beside water  
bank.n.02: financial institution  
bank.n.03: supply or stock held in reserve
```

Evaluation: Accuracy on sense-labeled test set

Challenge: Sense definitions subjective, hard to annotate

Q30. Explain relation extraction.

Answer: Identifying semantic relationships between entities.

Example:

```
"Steve Jobs founded Apple Inc."  
  
Entity 1: Steve Jobs (PERSON)  
Relation: founder_of  
Entity 2: Apple Inc. (ORGANIZATION)
```

Common relations:

- **founder_of**: Person → Organization
- **born_in**: Person → Location
- **works_for**: Person → Organization
- **capital_of**: City → Country
- **part_of**: Entity → Entity

Approaches:**1. Pattern-based:**

- Hand-crafted patterns
- "X founded Y" → founder_of(X, Y)

2. Feature-based ML:

- Extract features (lexical, syntactic)
- Train classifier

3. Neural:

- CNN, LSTM on sentence
- BERT with entity markers

```
"[E1] Steve Jobs [/E1] founded [E2] Apple [/E2]"
```

4. Distant supervision:

- Use KB to generate training data
- Noisy but scalable

Applications:

- Knowledge graph construction
- Question answering
- Information extraction

NLP Applications & Tools (Questions 31-40)

Q31. What is text generation and its challenges?

Answer: Creating coherent, fluent text automatically.

Types:**1. Unconditional:**

- Generate text from scratch

- Language model sampling

2. Conditional:

- Based on prompt/context
- Machine translation, summarization

Methods:

1. Rule-based:

- Templates
- Predictable, limited

2. Statistical:

- N-gram language models
- Markov chains

3. Neural:

- RNN/LSTM
- Transformers (GPT)

Decoding strategies:

- **Greedy:** Pick highest probability
- **Beam search:** Keep top-k sequences
- **Sampling:** Random based on distribution
- **Top-k/top-p:** Sample from top tokens

Challenges:

- **Repetition:** Models repeat phrases
- **Coherence:** Long-range consistency
- **Factuality:** Hallucinations
- **Diversity:** Avoiding generic responses
- **Control:** Steering generation

Evaluation:

- Perplexity
- BLEU (for task-specific)
- Human evaluation

Q32. Explain information extraction (IE).

Answer: Extracting structured information from unstructured text.

Tasks:

1. Named Entity Recognition:

"Apple Inc. was founded in California"
→ [Apple Inc.: ORG], [California: LOC]

2. Relation Extraction:

"Steve Jobs founded Apple"
→ (Steve Jobs, founder_of, Apple)

3. Event Extraction:

"Apple announced iPhone on Jan 9, 2007"
→ Event: announcement
Who: Apple
What: iPhone
When: Jan 9, 2007

4. Template Filling:

Acquisition template:
Acquirer: Microsoft
Acquired: LinkedIn
Amount: \$26.2B
Date: June 2016

Pipeline:

1. Text preprocessing
2. NER
3. Coreference resolution
4. Relation extraction
5. Event detection
6. Template filling

Applications:

- News analysis
- Financial analysis
- Scientific literature mining
- Legal document processing

Q33. What is semantic similarity?

Answer: Measuring how similar two pieces of text are in meaning.

Levels:**1. Lexical similarity:**

- Jaccard, edit distance
- Surface-level matching

2. Semantic similarity:

- Meaning-based
- "car" vs "automobile" (high similarity)

Methods:**1. WordNet-based:**

- Path similarity in hierarchy
- Leacock-Chodorow
- Wu-Palmer

2. Corpus-based:

- LSA (Latent Semantic Analysis)
- Word2Vec cosine similarity

3. Neural:

- Sentence-BERT
- Universal Sentence Encoder
- Cosine similarity of embeddings

Example:

```
S1: "The cat sat on the mat"
S2: "A feline rested on a rug"
Semantic similarity: High (similar meaning)
Lexical similarity: Low (different words)
```

Applications:

- Semantic search
- Duplicate detection
- Paraphrase detection
- Question matching

Evaluation: Correlation with human judgments (STS benchmark)

Q34. What is topic modeling?

Answer: Discovering abstract topics in document collections.

Popular method: LDA (Latent Dirichlet Allocation)**Assumptions:**

- Each document is mixture of topics
- Each topic is mixture of words

Example:

Topic 1 (Sports): game, team, player, score, win
Topic 2 (Politics): election, government, policy, vote
Topic 3 (Technology): software, computer, data, AI

Document: 50% Topic 1 + 30% Topic 2 + 20% Topic 3

Process:

1. Specify number of topics (k)
2. Initialize random topic assignments
3. Iteratively update assignments
4. Converge to topic distributions

Other methods:

- **LSA**: SVD on term-document matrix
- **NMF**: Non-negative Matrix Factorization
- **BERTopic**: BERT embeddings + clustering

Evaluation:

- Coherence score
- Perplexity
- Human interpretation

Applications:

- Document organization
- Trend analysis
- Recommendation
- Content discovery

Q35. Explain spell checking and correction.

Answer: Detecting and correcting spelling errors.

Error types:**1. Non-word errors:**

"teh" → "the" (not a valid word)

2. Real-word errors:

"I ate to much" → "I ate too much"
("to" is valid but wrong)

Approaches:

1. Dictionary lookup:

- Check if word exists
- Simple but limited

2. Edit distance:

- Minimum edits to reach valid word
- Levenshtein distance

"teh" → "the" (1 substitution)

3. Noisy channel model:

$$P(\text{correction}|\text{error}) \propto \underbrace{P(\text{error}|\text{correction})}_{\text{error model}} \times \underbrace{P(\text{correction})}_{\text{language model}}$$

4. Neural:

- Seq2seq models
- BERT for masked prediction
- Context-aware

Implementations:

- **Peter Norvig's algorithm**
- **Hunspell** (used in browsers)
- **SymSpell**
- **Neural spell checkers**

Challenges:

- Context dependency
- Names and rare words
- Multiple languages
- Homophones (their/there)

Q36. What is language detection?

Answer: Automatically identifying the language of text.

Approaches:

1. Character n-grams:

- Languages have characteristic patterns

```
English: "th", "he", "in"  
Spanish: "ón", "de", "la"
```

2. Word-based:

- Common words per language

```
English: "the", "is", "and"  
French: "le", "de", "un"
```

3. Machine Learning:

- Train classifier on labeled data
- Features: character n-grams, script

4. Neural:

- Character-level CNN/LSTM
- Language embeddings

Libraries:

- **langdetect** (Python): Google's library
- **langid**: Fast, accurate
- **fastText**: Meta's classifier

Challenges:

- Short texts
- Code-switching (mixed languages)
- Similar languages (Spanish/Catalan)
- Rare languages

Use cases:

- Content filtering
- Routing to translators
- Search engines
- Social media analytics

Q37. What is zero pronoun resolution?

Answer: Resolving omitted pronouns (common in pro-drop languages).

Example (Japanese):

"Yesterday went to store. Bought milk."

Implicit subject "I" in both sentences

Languages with zero pronouns:

- Japanese
- Chinese
- Spanish
- Italian
- Korean

Challenges:

- Ambiguity: who is the subject?
- Context dependency
- Cultural conventions

Approaches:

- Salience modeling
- Centering theory
- Neural models with context

English example:

"Want to go?"
(Implied "you" or "do you")

Importance: Critical for translation from pro-drop languages

Q38. What are sentence embeddings?

Answer: Fixed-size vector representations of entire sentences.

Methods:

1. Simple averaging:

- Average word embeddings

```
sentence_vec = mean([w1, w2, ..., wn])
```

- Loses word order

2. Doc2Vec/Sent2Vec:

- Extension of Word2Vec
- Learn sentence vectors directly

3. InferSent:

- BiLSTM trained on NLI data
- Good semantic representations

4. Universal Sentence Encoder (USE):

- Transformer or DAN
- Multilingual

5. Sentence-BERT (SBERT):

- BERT with siamese architecture
- Efficient similarity computation

```
from sentence_transformers import SentenceTransformer
model = SentenceTransformer('all-MiniLM-L6-v2')
embeddings = model.encode(sentences)
```

Properties:

- Semantically similar sentences close in vector space
- Fast similarity computation (cosine)

Applications:

- Semantic search
- Clustering
- Duplicate detection
- Recommendation

Q39. What is multilingual NLP?

Answer: Processing and understanding multiple languages.

Challenges:

1. Different scripts:

- Latin, Arabic, Chinese, Cyrillic, etc.

2. Different word orders:

- SVO (English), SOV (Japanese), VSO (Irish)

3. Morphological richness:

- Turkish, Finnish (complex word forms)

4. Resource imbalance:

- English: abundant resources
- Low-resource languages: limited data

Approaches:

1. Multilingual embeddings:

- Shared space for multiple languages
- mBERT, XLM-R, LaBSE

2. Zero-shot transfer:

- Train on high-resource
- Apply to low-resource

3. Cross-lingual alignment:

- Align embedding spaces

4. Massively multilingual models:

- mT5: 101 languages
- BLOOM: 46 languages
- NLLB: 200 languages

Tasks:

- Machine translation
- Cross-lingual search
- Multilingual classification
- Code-switching handling

Evaluation: XTREME benchmark

Q40. What is text augmentation?

Answer: Creating variations of training data to improve model robustness.

Techniques:

1. Synonym replacement:

```
"The movie was great"  
→ "The film was excellent"
```

2. Random insertion:

```
"The cat sat"  
→ "The small cat sat"
```

3. Random swap:

```
"I love NLP"  
→ "I NLP love"
```

4. Random deletion:

```
"The quick brown fox"  
→ "The brown fox"
```

5. Back-translation:

```
English → French → English  
"Hello world" → "Bonjour monde" → "Hello world"  
(slight variations)
```

6. Paraphrasing:

- Use paraphrase models
- T5, PEGASUS

7. Contextual word replacement:

- Use BERT to predict alternatives

```
"The [MASK] is blue" → sky, car, shirt
```

Benefits:

- Larger training set
- Better generalization
- Handles variations
- Reduces overfitting

Libraries: nlpaug, TextAttack

Practical NLP (Questions 41-50)

Q41. What are common NLP libraries and frameworks?

Answer:**Python libraries:****1. NLTK** (Natural Language Toolkit):

- Comprehensive, educational
- Tokenization, POS, parsing
- WordNet, corpora

2. spaCy:

- Industrial-strength
- Fast, production-ready
- NER, POS, dependency parsing

```
import spacy
nlp = spacy.load("en_core_web_sm")
doc = nlp("Apple is looking at buying U.K. startup")
```

3. Transformers (HuggingFace):

- Pre-trained models
- BERT, GPT, T5, etc.
- Easy fine-tuning

4. Gensim:

- Topic modeling
- Word2Vec, Doc2Vec
- Similarity queries

5. TextBlob:

- Simple API
- Sentiment, POS, translation

6. Stanford CoreNLP:

- Java-based
- Comprehensive pipeline
- State-of-the-art parsers

Frameworks:

- **AllenNLP**: Research framework
- **Flair**: Embedding library
- **FastText**: Efficient text classification

Q42. How do you handle imbalanced text classification?

Answer:

Problem: Unequal class distribution

```
Class A: 10,000 samples  
Class B: 100 samples (rare)
```

Solutions:

1. Resampling:

- **Oversample minority:** Duplicate rare examples
- **Undersample majority:** Remove common examples
- **SMOTE:** Synthetic minority samples

2. Class weights:

```
from sklearn.utils.class_weight import compute_class_weight  
weights = compute_class_weight('balanced', classes=np.unique(y), y=y)
```

3. Data augmentation:

- Augment minority class
- Synonym replacement, back-translation

4. Ensemble methods:

- Train multiple models on balanced subsets
- Combine predictions

5. Different metrics:

- Don't use accuracy
- Use F1, precision, recall, AUC-ROC
- Focus on minority class performance

6. Threshold adjustment:

- Lower threshold for minority class
- Adjust based on cost

7. Anomaly detection:

- If extremely imbalanced (1:1000)
- Treat as anomaly detection

Q43. What is the difference between extractive and abstractive summarization?

Answer:

Extractive Summarization:

- **Select** important sentences/phrases from original
- No new text generated
- Copy-paste approach

Example:

Original:

[S1] The conference was held in Paris.

[S2] It attracted 500 attendees.

[S3] Many topics were discussed.

[S4] The keynote was inspiring.

Extractive summary: [S1] [S2] [S4]

→ "The conference was held in Paris. It attracted 500 attendees. The keynote was inspiring."

Methods:

- TextRank (graph-based)
- TF-IDF scoring
- ML ranking models

Abstractive Summarization:

- **Generate** new sentences
- Paraphrase and condense
- Like human summarizing

Example:

Original: Same as above

Abstractive summary:

→ "A 500-attendee Paris conference featured an inspiring keynote."

Methods:

- Seq2seq models
- Transformer models (BART, T5, Pegasus)
- GPT-based

Comparison:

Aspect	Extractive	Abstractive
Fluency	Can be choppy	More natural

Aspect	Extractive	Abstractive
Factuality	High (copy original)	Risk of hallucination
Compression	Limited	High
Complexity	Simpler	Complex
Coherence	May lack	Better

Q44. Explain the evaluation metrics for NLP tasks.

Answer:

Classification metrics:

- **Accuracy:** $(TP+TN)/(TP+TN+FP+FN)$
- **Precision:** $TP/(TP+FP)$
- **Recall:** $TP/(TP+FN)$
- **F1:** $2 \times (P \times R)/(P+R)$
- **AUC-ROC:** Area under ROC curve

Sequence labeling (NER, POS):

- Token-level accuracy
- Span-level F1
- Entity-level F1 (exact match)

Machine Translation:

- **BLEU:** N-gram precision with brevity penalty
- **METEOR:** Handles synonyms, stemming
- **chrF:** Character n-grams
- **COMET:** Neural metric

Summarization:

- **ROUGE-N:** N-gram recall
- **ROUGE-L:** Longest common subsequence
- **BERTScore:** Semantic similarity

Question Answering:

- **Exact Match:** Binary (correct/incorrect)
- **F1:** Token overlap

Language Modeling:

- **Perplexity:** $\exp(-\text{avg log likelihood})$
- Lower is better

General:

- **Human evaluation:** Gold standard
- **Correlation with humans:** Metric validation

Important: Choose metrics aligned with task goals

Q45. What are common NLP preprocessing steps?

Answer:

Pipeline:

1. Text cleaning:

```
# Remove HTML tags
clean_text = re.sub('<.*?>', '', text)

# Remove URLs
clean_text = re.sub(r'http\S+', '', text)

# Remove special characters
clean_text = re.sub(r'^a-zA-Z0-9\s]', '', text)
```

2. Tokenization:

```
from nltk.tokenize import word_tokenize
tokens = word_tokenize(text)
```

3. Lowercasing:

```
tokens = [t.lower() for t in tokens]
```

4. Stop word removal:

```
from nltk.corpus import stopwords
stop_words = set(stopwords.words('english'))
filtered = [t for t in tokens if t not in stop_words]
```

5. Stemming/Lemmatization:

```
from nltk.stem import WordNetLemmatizer
lemmatizer = WordNetLemmatizer()
lemmatized = [lemmatizer.lemmatize(t) for t in tokens]
```

6. Handling numbers:

```
# Remove or replace
tokens = [t for t in tokens if not t.isdigit()]
# Or normalize
tokens = ['NUM' if t.isdigit() else t for t in tokens]
```

Task-dependent:

- Sentiment: Keep negations, punctuation
- NER: Keep capitalization
- Topic modeling: Remove stop words
- Modern transformers: Minimal preprocessing

Q46. How do you handle out-of-vocabulary (OOV) words?

Answer:

Problem: Words not in training vocabulary

Solutions:

1. Subword tokenization:

- BPE, WordPiece, SentencePiece
- Break unknown words into subwords

```
"unhappiness" → ["un", "happi", "ness"]
```

2. Character-level models:

- Use character embeddings
- Can handle any word

3. Fallback tokens:

- Replace with **<UNK>** token

```
"I saw a quokka" → "I saw a <UNK>"
```

4. Hashing trick:

- Hash word to fixed vocabulary index

5. FastText:

- Character n-gram embeddings

- Generate embeddings for OOV words

6. Contextualized embeddings:

- BERT, GPT handle via subword
- Generate contextual representations

7. Back-off strategies:

- Use similar word
- Use word shape features

Modern approach: Subword tokenization (most effective)

Example (BERT):

```
"biodegradable" → ["bio", "##de", "##grad", "##able"]
```

Q47. What is domain adaptation in NLP?

Answer: Adapting models trained on one domain to perform well on another.

Problem:

```
Source: News articles  
Target: Medical records  
→ Different vocabulary, style, topics
```

Approaches:

1. Transfer learning:

- Pre-train on source
- Fine-tune on target

```
BERT (general) → Fine-tune on medical texts → BioBERT
```

2. Domain-specific pre-training:

- Continue pre-training on target domain
- Adapt vocabulary and representations

3. Multi-task learning:

- Train on both domains simultaneously

4. Data augmentation:

- Generate synthetic target domain data

5. Adversarial training:

- Learn domain-invariant features

6. Few-shot adaptation:

- Adapt with few target examples

Examples:

- **SciBERT**: Scientific papers
- **BioBERT**: Biomedical
- **FinBERT**: Financial
- **LegalBERT**: Legal documents

Challenges:

- Limited target data
- Domain shift
- Vocabulary mismatch

Q48. What are common challenges in production NLP systems?

Answer:

Challenges:

1. Latency:

- Large models (BERT) slow
- Solutions: Distillation, quantization, caching

2. Scalability:

- High traffic
- Solutions: Batching, load balancing, GPU clusters

3. Model size:

- BERT-large: 1.3GB
- Solutions: Compression, distillation, pruning

4. Data drift:

- Language evolves, new topics
- Solutions: Monitoring, periodic retraining

5. Multilingual support:

- Different models per language expensive
- Solutions: Multilingual models (mBERT)

6. Reproducibility:

- Random seeds, library versions
- Solutions: Version control, containers (Docker)

7. Bias and fairness:

- Models learn societal biases
- Solutions: Bias audits, debiasing techniques

8. Explainability:

- Black box models
- Solutions: Attention visualization, LIME, SHAP

9. Error handling:

- Graceful degradation
- Fallback mechanisms

10. Cost:

- GPU inference expensive
- Solutions: CPU optimization, model sharing

Best practices:

- Monitor metrics
- A/B testing
- Gradual rollout
- Version control

Q49. What is conversational AI architecture?

Answer:

Components:**1. Speech Recognition (ASR):**

- Audio → Text
- Whisper, Wav2Vec

2. Natural Language Understanding (NLU):

- **Intent classification:**

```
"Book a flight" → intent: book_flight
```

- **Entity extraction:**

```
"to NYC tomorrow" → {destination: NYC, date: tomorrow}
```

3. Dialogue Management:

- **Dialogue state tracking:** Current conversation state
- **Policy:** Decide next action
- **Context management:** Remember conversation history

4. Natural Language Generation (NLG):

- Generate response text
- Template-based or neural (GPT)

5. Text-to-Speech (TTS):

- Text → Audio
- Tacotron, FastSpeech

Architecture:

```

User → ASR → NLU → Dialogue Manager → NLG → TTS → User
                ↑           ↓
            Context   KB/API
  
```

Modern approach:

- End-to-end neural (GPT-based)
- RAG for knowledge
- Function calling for actions

Examples: Alexa, Siri, Google Assistant, ChatGPT

Q50. What are recent trends and future directions in NLP?

Answer:

Current trends:

1. Large Language Models:

- GPT-4, Claude, Gemini
- Emergent abilities at scale
- In-context learning

2. Multimodal models:

- Text + images (GPT-4V, Gemini)
- Text + audio + video
- Unified representations

3. Efficient models:

- Distillation (DistilBERT)
- Quantization (GPTQ, GGUF)
- Sparse models (Mixture of Experts)

4. Retrieval augmentation:

- RAG for factuality
- External knowledge integration

5. Alignment and safety:

- RLHF
- Constitutional AI
- Red teaming

6. Code generation:

- Copilot, CodeLlama
- Software engineering automation

Future directions:**1. Smaller, efficient models:**

- On-device LLMs
- Specialized models

2. Better reasoning:

- Chain-of-thought
- Tool use
- Planning

3. Multilinguality:

- 1000+ languages support
- Low-resource languages

4. Continual learning:

- Update without full retrain
- Adapt to new information

5. Interpretability:

- Understanding model decisions
- Mechanistic interpretability

6. Domain-specific LLMs:

- Medical, legal, scientific
- High accuracy needs

Quick Interview Tips

1. **Know the basics:** Tokenization, embeddings, transformers
2. **Hands-on experience:** Mention projects with spaCy, NLTK, Transformers
3. **Stay current:** Know GPT, BERT, recent advances
4. **Evaluation:** Understand metrics for each task
5. **Preprocessing:** Know when to apply which steps
6. **Trade-offs:** Discuss accuracy vs speed, size vs performance
7. **Applications:** Relate concepts to real-world uses

Good luck with your NLP interview! 🚀